Riya Varyani
D20A - 61

Experiment.4

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Theory:

Solidity and Smart Contract Fundamentals
1. Primitive Data Types, Variables, and Functions (pure, view)

Solidity is a high-level, statically typed programming language used for developing smart contracts on the Ethereum blockchain. Primitive data types serve as the fundamental building blocks of Solidity and play a crucial role in defining variables, performing computations, and managing data within a contract.

Commonly used primitive data types include:

- **uint / int**: Used to represent unsigned and signed integers of different sizes, such as uint256 or int128. These are widely used for financial calculations, counters, and token balances.
- **bool**: Represents logical values (true or false) and is commonly used in decision-making statements such as access control and validations.
- **address**: Stores a 20-byte Ethereum account or contract address, which is essential for identifying users and interacting with other contracts.
- **bytes / string**: Used for storing binary and textual data, respectively, such as user names or encrypted information.

Variables in Solidity are categorized into three types based on their scope and lifetime:

- **State Variables**: Stored permanently on the blockchain and consume gas when modified. Example: storing the contract owner's address.
- **Local Variables**: Temporary variables that exist only during function execution and are not stored on-chain.
- **Global Variables**: Predefined variables such as msg.sender, msg.value, and block.timestamp, which provide information about the transaction and blockchain environment.

Functions in Solidity define the logic and behavior of smart contracts. Two important function types include:

- **pure**: Functions that neither read nor modify blockchain state; they operate only on input values. Example: a function that adds two numbers.
- **view**: Functions that can read state variables but cannot modify them. Example: a function that returns an account balance.

These function types help in optimizing gas usage and ensuring secure and predictable contract execution.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input parameters and return output values, enabling interaction between users and smart contracts. Inputs allow external users or other contracts to pass data into a function, while outputs enable the function to return computed results.

For example, a function may accept an amount of Ether as input and return a boolean value indicating whether a transaction was successful. Solidity also supports named return variables, which enhance code readability and simplify debugging.

3. Visibility, Modifiers, and Constructors

Function visibility defines who can access a particular function within or outside the contract:

- **public**: Accessible both internally and externally.
- **private**: Accessible only within the same contract.
- **internal**: Accessible within the contract and its derived contracts.
- **external**: Can only be called by external accounts or other contracts.

Modifiers are special functions that modify the behavior of other functions. They are commonly used for security and access control. For instance, an onlyOwner modifier can restrict certain functions so that only the contract owner can execute them.

A **constructor** is a special function that executes only once during contract deployment. It is primarily used to initialize important variables, such as assigning the deploying account as the contract owner.

4. Control Flow: if-else and Loops

Solidity supports standard control flow structures similar to traditional programming languages.

- **if-else statements** allow conditional decision-making in contract logic. For example, a contract can verify whether a user has sufficient balance before processing a transaction.

- **Loops (for, while, do-while)** enable repeated execution of code, such as iterating through an array of registered users. However, loops must be used carefully, as excessive iterations increase gas consumption and may make transactions costly or even fail due to gas limits.

5. Data Structures: Arrays, Mappings, Structs, and Enums

Solidity provides several data structures to efficiently manage and organize data within smart contracts.

- **Arrays**: Used to store ordered collections of elements. They can be fixed-size or dynamic. Example: an array storing addresses of registered users.
- **Mappings**: Key-value storage structures that allow fast data retrieval. Example: mapping(address => uint) to store user balances. Unlike arrays, mappings do not support iteration.

Structs: Allow grouping of related data into a single entity. Example:

```
 struct Player {
string name;
uint score; }
```

- This structure helps store multiple attributes of a player in an organized manner.

Enums: Used to define a set of predefined constants, improving code clarity and readability. Example:

```
 enum Status { Pending, Active, Closed }
```

6. Data Locations

Solidity defines three primary data locations that determine where variables are stored and how they behave in memory:

- storage: Permanent data stored on the blockchain. Used for state variables.
- memory: Temporary data that exists only during function execution. Used for local variables and function inputs.
- calldata: A non-modifiable and non-persistent location used for external function parameters. It is more gas-efficient than memory.

Understanding data locations is essential because they directly impact gas costs, efficiency, and contract performance.

7. Transactions: Ether, Wei, Gas, and Sending Transactions

Ether and Wei:
Ether is the native cryptocurrency of the Ethereum blockchain. The smallest unit of Ether is Wei, where:
1 Ether = 10^{18} Wei.
This ensures high precision in financial and smart contract transactions.

Gas and Gas Price:
Every transaction on Ethereum consumes gas, which represents the computational cost of executing operations. The gas price determines how much Ether is paid per unit of gas. A higher gas price results in faster transaction processing.

Sending Transactions:
Transactions are used to transfer Ether or interact with smart contracts. Common methods include:

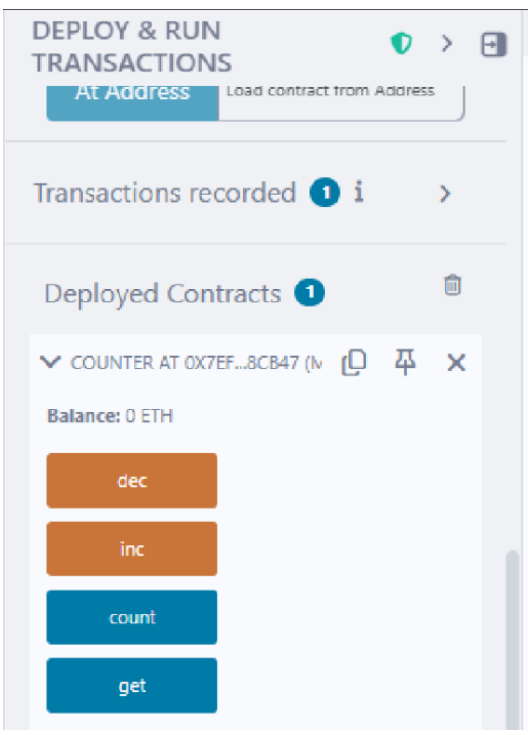
- transfer() – Safe but limited in gas usage.
- send() – Returns a boolean value indicating success or failure.
- call() – More flexible and recommended for modern Solidity development.

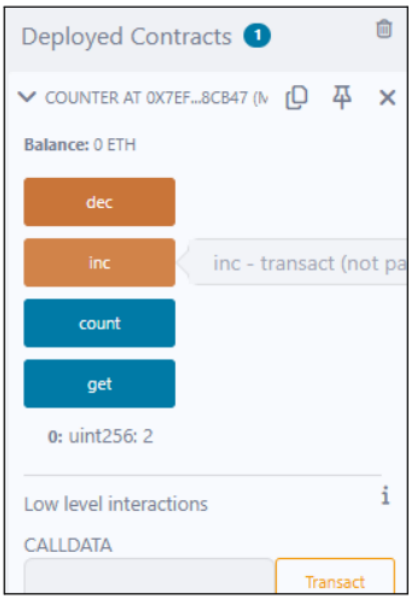
Efficient contract design is necessary to minimize gas consumption and reduce transaction costs.

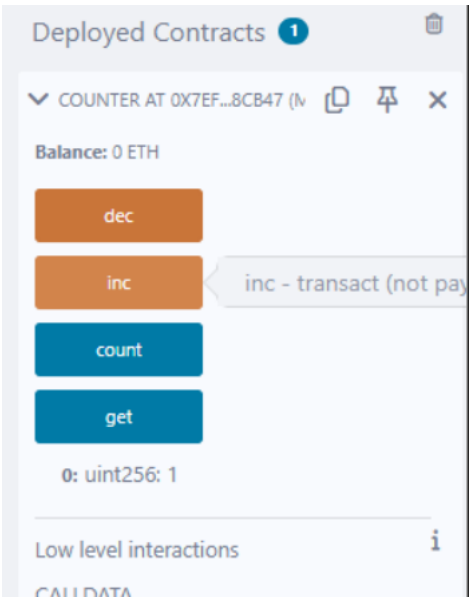
Output -

1. Introduction

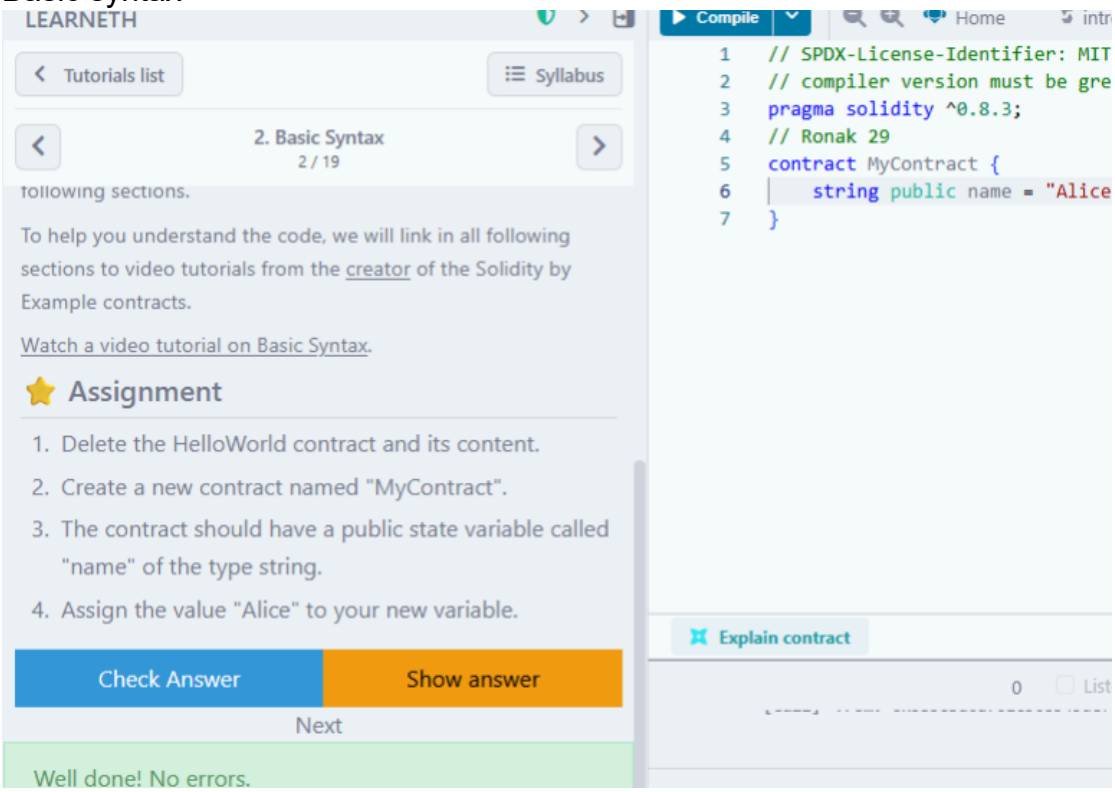
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;

contract Counter {
    uint public count;

    // Function to get the current count
    function get() public view returns (uint) {    2453 gas
        return count;
    }
    // Riya D20 A 61

    // Function to increment count by 1
    function inc() public {    infinite gas
        count += 1;
    }

    // Function to decrement count by 1
    function dec() public {    infinite gas
        count -= 1;
    }
}
```



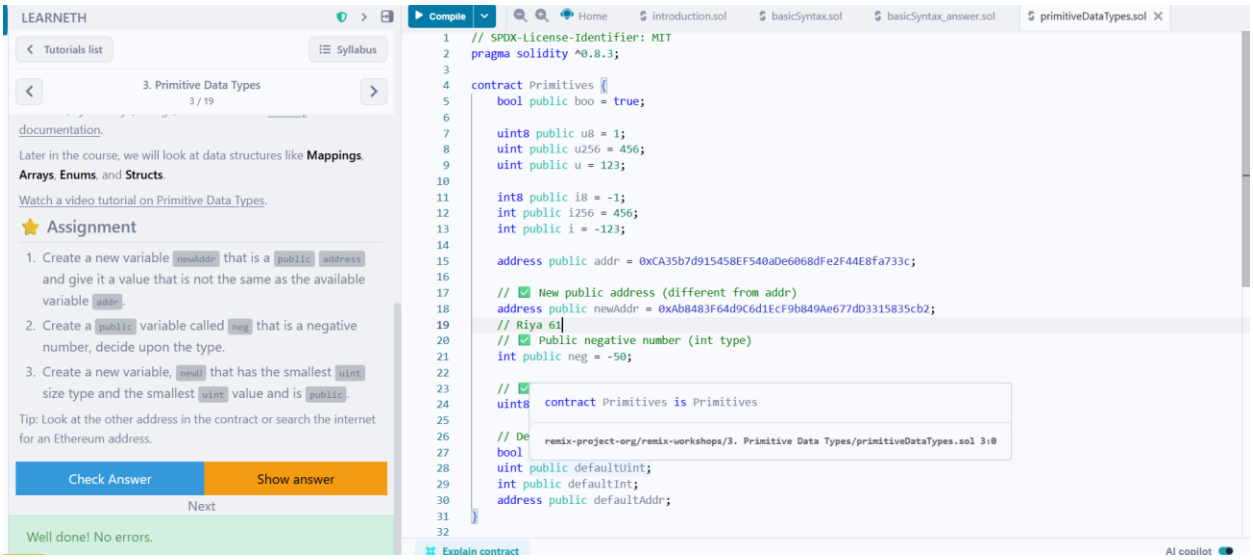




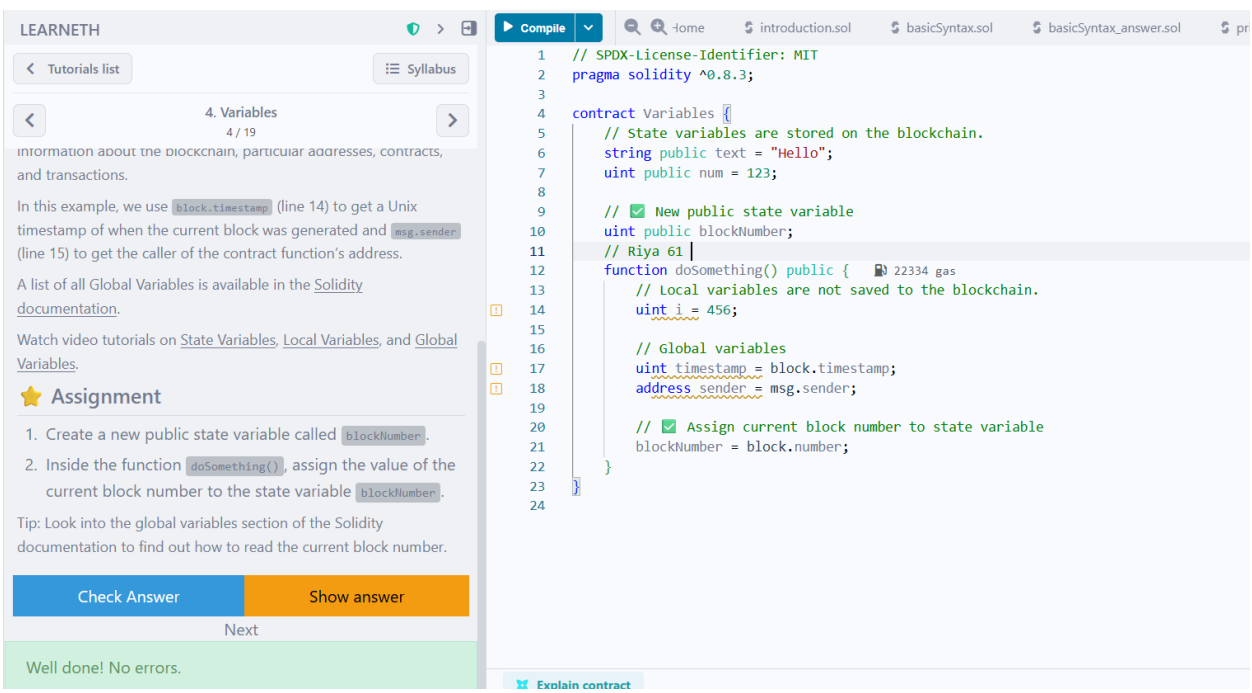
2. Basic syntax



3. primitiveDataTypes.sol


LEARNETH
Tutorials list
Syllabus
3. Primitive Data Types
3 / 19
documentation.
Later in the course, we will look at data structures like Mappings, Arrays, Enums, and Structs.
Watch a video tutorial on Primitive Data Types.
Assignment
1. Create a new variable newAddr that is a public address and give it a value that is not the same as the available variable addr.
2. Create a public variable called neg that is a negative number, decide upon the type.
3. Create a new variable, newU that has the smallest uint size type and the smallest uint value and is public.
Tip: Look at the other address in the contract or search the internet for an Ethereum address.
Check Answer
Show answer
Next
Well done! No errors.
Compile
Home
introduction.sol
basicSyntax.sol
basicSyntax_answer.sol
primitiveDataTypes.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Primitives {
bool public boo = true;
uint8 public u8 = 1;
uint public u256 = 456;
uint public u = 123;
int8 public i8 = -1;
int public i256 = 456;
int public i = -123;
address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;
// New public address (different from addr)
address public newAddr = 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2;
// Riya 61
// Public negative number (int type)
int public neg = -50;
contract Primitives is Primitives
remix-project-org/remix-workshops/3. Primitive Data Types/primitiveDataTypes.sol 3:0
uint public defaultUint;
int public defaultInt;
address public defaultAddr;
}
Explain contract
AI copilot

4. Variables


LEARNETH
Tutorials list
Syllabus
4. Variables
4 / 19
information about the blockchain, particular addresses, contracts, and transactions.
In this example, we use block.timestamp (line 14) to get a Unix timestamp of when the current block was generated and msg.sender (line 15) to get the caller of the contract function's address.
A list of all Global Variables is available in the Solidity documentation.
Watch video tutorials on State Variables, Local Variables, and Global Variables.
Assignment
1. Create a new public state variable called blockNumber.
2. Inside the function doSomething(), assign the value of the current block number to the state variable blockNumber.
Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.
Check Answer
Show answer
Next
Well done! No errors.
Compile
Home
introduction.sol
basicSyntax.sol
basicSyntax_answer.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Variables {
// State variables are stored on the blockchain.
string public text = "Hello";
uint public num = 123;
// New public state variable
uint public blockNumber;
// Riya 61
function doSomething() public {
22334 gas
// Local variables are not saved to the blockchain.
uint i = 456;
// Global variables
uint timestamp = block.timestamp;
address sender = msg.sender;
// Assign current block number to state variable
blockNumber = block.number;
}
}
Explain contract

5.1 readAndWrite.sol

LEARNETH

Tutorials list | Syllabus

5.1 Functions - Reading and Writing to a State Variable
5 / 19

use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on Functions.

Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer | Show answer

Next

Well done! No errors.

Compile | basicSyntax.sol | basicSyntax_answer.sol | primitiveDataTypes.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;

contract SimpleStorage {
    // State variable to store a number
    uint public num;

    // ✅ Public boolean state variable initialized to true
    bool public b = true;

    // Write to a state variable
    function set(uint _num) public {    22536 gas
        num = _num;
    }
    // Riya 61
    // Read from a state variable
    function get() public view returns (uint) {    2475 gas
        return num;
    }

    // ✅ Public function that returns the value of b
    function get_b() public view returns (bool) {    2539 gas
        return b;
    }
}
```

Explain contract

5.2 view and pure

LEARNETH

Tutorials list | Syllabus

5.2 Functions - View and Pure
6 / 19

From the Solidity documentation.

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions view and pure can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer | Show answer

Next

Well done! No errors.

Compile | basicSyntax_answer.sol | primitiveDataTypes.sol | variables.sol 3 | readAndWr

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract ViewAndPure {
5      uint public x = 1;
6
7      // Promise not to modify the state.
8      function addToX(uint y) public view returns (uint) {    infinite gas
9          return x + y;
10     }
11     //Riya 61
12     // ✅ Updates the state variable x
13     function addToX2(uint y) public {    infinite gas
14         x = x + y;
15     }
16
17     // Promise not to modify or read from the state.
18     function add(uint i, uint j) public pure returns (uint) {    infinite gas
19         return i + j;
20     }
21 }
22
```

Explain contract

5.3 modifiersAndConstructor.sol

LEARNETH

Tutorials list | Syllabus

5.3 Functions - Modifiers and Constructors
7 / 19

A constructor function is executed upon the creation of a contract. You can use it to run contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that x can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer | Show answer

Next

Well done! No errors.

Compiled | ure.sol | viewAndPure_answer.sol | modifiersAndConstructors.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract FunctionModifier {
5      // We will use these variables to demonstrate how to use
6      // modifiers.
7      address public owner;
8      uint public x = 10;
9      bool public locked;
10
11     constructor() {    461217 gas 414400 gas
12         // Set the transaction sender as the owner of the contract.
13         owner = msg.sender;
14     }
15
16     // Modifier to check that the caller is the owner of
17     // the contract.
18     modifier onlyOwner() {
19         require(msg.sender == owner, "Not owner");
20         // Underscore is a special character only used inside
21         // a function modifier and it tells Solidity to
22         // execute the rest of the code.
23         _;
24     }
25
26     // Modifiers can take inputs. This modifier checks that the
27     // address passed in is not the zero address.
28     modifier validAddress(address _addr) {
29         require(_addr != address(0), "Not valid address");
30         _;
31     }
32
```

5.3 Modifiers and Construction

LEARNETH

Tutorials list | Syllabus

5.3 Functions - Modifiers and Constructors
7 / 19

have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that x can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer | Show answer

Next

Well done! No errors.

Compile | primitiveDataTypes.sol | variables.sol 3 | readAndWrite.sol | viewAndPure.sol

```
25         onlyOwner
26         validAddress(_newOwner)
27     {
28         owner = _newOwner;
29     }
30
31     modifier noReentrancy() {
32         require(!locked, "No reentrancy");
33         locked = true;
34         _;
35         locked = false;
36     }
37
38     function decrement(uint i) public noReentrancy {    infinite gas
39         x -= i;
40
41         if (i > 1) {
42             decrement(i - 1);
43         }
44     }
45     //Riya 61
46     // ✅ Modifier that increases x
47     modifier increase(uint _value) {
48         x += _value;
49         _;
50     }
51
52     // ✅ Function with EMPTY body
53     function increaseX(uint _value) public increase(_value) {}    infinite gas
54 }
55
```

Explain contract

5.4 Inputs and Outputs

LEARNETH

Tutorials list | Syllabus

5.4 Functions - Inputs and Outputs
8 / 19

output parameters of contract functions.

"*[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible.*" From the Solidity documentation.

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer | Show answer

Next

Well done! No errors.

Compile | variables.sol 3 | readAndWrite.sol | viewAndPure.sol | modifiersAnd

```
30
31      function destructingAssigments()    infinite gas
32          public
33          pure
34          returns (uint, bool, uint, uint, uint)
35      {
36          (uint i, bool b, uint j) = returnMany();
37          (uint x, , uint y) = (4, 5, 6);
38
39          return (i, b, j, x, y);
40      }
41      //  Riya 61
42      // ✅ New function
43      function returnTwo()    472 gas
44          public
45          pure
46          returns (int x, bool b)
47      {
48          x = -2;
49          b = true;
50      }
51
52      function arrayInput(uint[] memory _arr) public {}    infinite gas
53
54      uint[] public arr;
55
56      function arrayOutput() public view returns (uint[] memory) {    infinite gas
57          return arr;
58      }
59  }
60
```

Explain contract

6. visibility.sol

LEARNETH

Tutorials list | Syllabus

6. Visibility
9 / 19

- State variables can not be `external`

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on Visibility.

Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer | Show answer

Next

Well done! No errors.

Compile | readAndWrite.sol | viewAndPure.sol | modifiersAndConstructors.sol | inputsAndOut

```
22          return "public function called";
23      }
24
25      function externalFunc() external pure returns (string memory) {    infinite gas
26          return "external function called";
27      }
28
29      // State variables
30      string private privateVar = "my private variable";
31      string internal internalVar = "my internal variable";
32      string public publicVar = "my public variable";
33  }
34
35  contract Child is Base {
36      function testInternalFunc() public pure override returns (string memory) {    infinite gas
37          return internalFunc();
38      }
39      //Riya 61
40      // ✅ New function
41      function testInternalVar()    infinite gas
42          public
43          view
44          returns (string memory, string memory)
45      {
46          return (internalVar, publicVar);
47      }
48  }
49
```

Explain contract

7.1 Control Flow

LEARNETH

Tutorials list | Syllabus

7.1 Control Flow - If/Else
10 / 19

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the foo function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternery operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer | Show answer

Next

Well done! No errors.

Compile | ite.sol | viewAndPure.sol | modifiersAndConstructors.sol | inputsA

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract IfElse {
5      function foo(uint x) public pure returns (uint) {    infinite gas
6          if (x < 10) {
7              return 0;
8          } else if (x < 20) {
9              return 1;
10         } else {
11             return 2;
12         }
13     }
14
15     function ternary(uint _x) public pure returns (uint) {    infinite gas
16         return _x < 10 ? 1 : 2;
17     }
18
19     // ✅ New function
20     function evenCheck(uint x) public pure returns (bool) {    infinite gas
21         return x % 2 == 0 ? true : false;
22     }
23 }
24
```

8.1 Data Structures

LEARNETH

Tutorials list | Syllabus

8.1 Data Structures - Arrays
12 / 19

with the `delete` operator all other elements stay the same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer | Show answer

Next

Well done! No errors.

Compile | :tors.sol | inputsAndOutputs.sol | visibility.sol | ifElse.sol

```
10     uint[10] public myFixedSizeArr;
11
12     // ✅ New fixed-size array with smallest size
13     uint[3] public arr3 = [0, 1, 2];
14
15     function get(uint i) public view returns (uint) {    infinite gas
16         return arr[i];
17     }
18     // Riya 61
19     // ✅ Return arr3 instead of arr
20     function getArr() public view returns (uint[3] memory) {    infinite gas
21         return arr3;
22     }
23
24     function push(uint i) public {    46820 gas
25         arr.push(i);
26     }
27
28     function pop() public {    29462 gas
29         arr.pop();
30     }
31
32     function getLength() public view returns (uint) {    2483 gas
33         return arr.length;
34     }
35
36     function remove(uint index) public {    7609 gas
37         delete arr[index];
38     }
39 }
40
```

8.3 Structs

LEARNETH

Tutorials list | Syllabus

8.3 Data Structures - Structs
14 / 19

Key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19).

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a structs' member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer | Show answer

Next

Well done! No errors.

Compile | visibility.sol | ifElse.sol | loops.sol | loops_answer.sol

```
15
16          Todo memory todo;
17          todo.text = _text;
18          todos.push(todo);
19      }
20
21      function get(uint _index)    infinite gas
22          public
23          view
24          returns (string memory text, bool completed)
25      {
26          Todo storage todo = todos[_index];
27          return (todo.text, todo.completed);
28      }
29
30      function update(uint _index, string memory _text) public {    infinite ga
31          Todo storage todo = todos[_index];
32          todo.text = _text;
33      }
34
35      function toggleCompleted(uint _index) public {    28995 gas
36          Todo storage todo = todos[_index];
37          todo.completed = !todo.completed;
38      }
39      //Riya 61
40      // ✅ New remove function
41      function remove(uint _index) public {    infinite gas
42          delete todos[_index];
43      }
44  }
45
```

8.4 Enums

Tutorials list | Syllabus

8.4 Data Structures - Enums
15 / 19

representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer | Show answer

Next

Well done! No errors.

```
23      // ✅ Enum variable initialized (defaults to first value: S)
24      Size public sizes;
25
26      function get() public view returns (Status) {    2605 gas
27          return status;
28      }
29
30      function set(Status _status) public {    undefined gas
31          status = _status;
32      }
33
34      function cancel() public {    24494 gas
35          status = Status.Canceled;
36      }
37
38      function reset() public {    24383 gas
39          delete status;
40      }
41      //Riya 61
42      // ✅ Getter function for Size enum
43      function getSize() public view returns (Size) {    2633 gas
44          return sizes;
45      }
46  }
47
```

Explain contract

9.Data Locations

LEARNETH

Tutorials list | Syllabus

9. Data Locations
16 / 19

Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location *memory* inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location *memory* inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function f return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer | Show answer

Next

Well done! No errors.

Compile | loops.sol | loops_answer.sol | arrays.sol | mappings.sol | structs.sol

```
29        MyStruct memory myMemStruct = MyStruct(0);
30
31        // ✅ Create myMemStruct2 from myMemStruct
32        MyStruct memory myMemStruct2 = myMemStruct;
33        myMemStruct2.foo = 1;
34
35        // ✅ Create myMemStruct3 from myStruct (storage → memory)
36        MyStruct memory myMemStruct3 = myStruct;
37        myMemStruct3.foo = 3;
38
39        // ✅ Return all three
40        return (myStruct, myMemStruct2, myMemStruct3);
41    }
42    // Riya Varyani
43    function _f(    undefined gas
44        uint[] storage _arr,
45        mapping(uint => address) storage _map,
46        MyStruct storage _myStruct
47    ) internal {}
48
49    function g(uint[] memory _arr) public returns (uint[] memory) {    infinite gas
50        return _arr;
51    }
52
53    function h(uint[] calldata _arr) external {}    468 gas
54
55
```

10.1Ether and Wei

LEARNETH

Tutorials list | Syllabus

10.1 Transactions - Ether and Wei
17 / 19

Wei is the smallest subunit of *Ether*, named after the cryptographer Wei Dai. *Ether* numbers without a suffix are treated as `wei` (line 7).

`gwei`

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`ether`

One `ether` is equal to 1,000,000,000,000,000,000 (10^18) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

Assignment

1. Create a `public` `uint` called `oneGWei` and set it to 1 `gwei`.
2. Create a `public` `bool` called `isOneGWei` and set it to the result of a comparison operation between 1 gwei and 10^9.

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer | Show answer

Next

Well done! No errors.

Compile | er.sol | arrays.sol | mappings.sol | structs.sol | en

```
1   // SPDX-License-Identifier: MIT
2   pragma solidity ^0.8.3;
3
4   contract EtherUnits {
5       uint public oneWei = 1 wei;
6       bool public isOneWei = 1 wei == 1;
7
8       // ✅ Correct naming
9       uint public oneGwei = 1 gwei;
10      bool public isOneGwei = 1 gwei == 1e9;
11
12      uint public oneEther = 1 ether;
13      bool public isOneEther = 1 ether == 1e18;
14  }
15
```

Explain contract

Tutorials list | Syllabus

10.3 Transactions - Sending Ether

19 / 19

Watch a video tutorial on Sending Ether.

Assignment

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

1. Create a contract called `Charity`.
2. Add a public state variable called `owner` of the type address.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer | Show answer

Next

Well done! No errors.

```
receive() exists?  fallback()
        /   \
       yes   no
       /      \
    receive()   fallback()
    */

    // Beneficiary of the charity
    address public owner;

    constructor() {    106591 gas 82200 gas
        owner = msg.sender;
    }

    // Function to receive Ether donations
    function donate() public payable {}    141 gas
    //Riya 61
    // Withdraw total balance to the owner
    function withdraw() public {    infinite gas
        payable(owner).transfer(address(this).balance);
    }
}
```